

M05 - Python for Data Engineering

15-lesson learner book | Beginner-first professional institute standard | B01-style teaching contract

Purpose

Use Python for files, pandas, APIs, PostgreSQL, validation, logging and reusable pipeline code.

Course boundary

This is the independent Standalone Data Engineer course, not Four-Course C3. Complete the course in sequence; each new idea is still taught from first principles.

Option B institute identity | Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6

This book is your teacher

Use one lesson at a time. Do not read protected review before saving a genuine attempt. Every lesson follows the same beginner-first sequence even when the engineering idea becomes more technical.

Daily lesson loop

1) why it matters -> 2) what you learn -> 3) prerequisite -> 4) picture it -> 5) new words -> 6) useful visual/source -> 7) follow once -> 8) expected result -> 9) why it worked -> 10) mistakes -> 11) recovery -> 12) independent task -> 13) validate -> 14) teach it back -> 15) protected review + changed retry -> 16) evidence/mastery + exact next route.

Prerequisite check

Complete M04 Python Foundations. You should run scripts, use functions/collections, handle files/CSV/JSON, and debug basic exceptions.

Attempt protection

A protected review is repair material, not a shortcut. Save the first attempt, open only the matching review if needed, record the repair, close review, then solve a changed retry from a clean state.

Contents / Index

Every formal lesson and its actual page number in this exact rebuilt book.

DE05-01 - CSV properly: parse, inspect, preserve raw	5
DE05-02 - JSON and JSONL from zero	7
DE05-03 - Paths, folders and deterministic batch files	9
DE05-04 - pandas profiling before transformation	11
DE05-05 - Cleaning with pandas and rejected rows	13
DE05-06 - pandas merge, cardinality and aggregation	15
DE05-07 - Data validation: record, batch and relationship rules	17
DE05-08 - HTTP and REST APIs from zero	19
DE05-09 - Calling APIs safely with requests	21
DE05-10 - Pagination and authentication without leaking secrets	23
DE05-11 - Retries, rate limits and partial failure	25
DE05-12 - Python + PostgreSQL with SQLAlchemy	27
DE05-13 - Configuration, environment variables and secret hygiene	29
DE05-14 - Logging, testing and reusable pipeline code	31
DE05-15 - Integrated Python data-engineering pipeline	33

Prerequisite, tool and outcome map

ASSUMED

Complete M04 Python Foundations. You should run scripts, use functions/collections, handle files/CSV/JSON, and debug basic exceptions.

MODULE PURPOSE

Use Python for files, pandas, APIs, PostgreSQL, validation, logging and reusable pipeline code.

EXIT EVIDENCE

Module practical + fresh retest + saved evidence; no critical unresolved failure.

Protected study route

Teaching -> assigned visual/source when useful -> follow once -> close worked material -> independent attempt -> validate -> save evidence -> teach it back. If weak: matching protected review -> targeted repair -> close review -> changed fresh retry -> revalidate.

DE05-01 - CSV properly: parse, inspect, preserve raw

1. Why it matters

Parse CSV deliberately, inspect its contract, and preserve raw input before transformation. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Parse CSV deliberately, inspect its contract, and preserve raw input before transformation.

3. Prerequisite / what you already know

Complete M04 Python Foundations. You should run scripts, use functions/collections, handle files/CSV/JSON, and debug basic exceptions.

4. Picture it in your head

Picture raw CSV as sealed evidence. First record its identity and controls, then copy it into a preserved raw area; transformations happen on downstream copies, never by overwriting the received source.

5. New words before we use them

A CSV file is text with a tabular convention, not a magically typed table. Preserve the received bytes or immutable source copy so later

- debugging can distinguish source from transformation.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Inspect orders_good.csv. Save raw row count, field names and first/last order_id. Copy it to an immutable raw_received folder without editing.

8. Expected result / what you should see

Row count and field list are explicit; raw copy hash equals source hash; no transformation overwrites raw.

9. Why it worked

The workflow is trustworthy because the received input is identified and preserved before transformation, so every downstream result can be traced back to unchanged source evidence. Your independent check is: For CSV properly: parse, inspect, preserve raw: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling CSV properly: parse, inspect, preserve raw complete because the command or tool finished. 2. Skipping the independent control: For CSV properly: parse, inspect, preserve raw: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture raw CSV as sealed evidence. 3. Run the earliest independent control: For CSV properly: parse, inspect, preserve raw: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Given orders_new.csv, produce a source receipt note and prove the preserved raw copy is byte-identical to the supplied source.

13. Validate your result

For CSV properly: parse, inspect, preserve raw: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

14. Teach it back in your own words

Without notes, explain CSV properly: parse, inspect, preserve raw in your own words. Start with the purpose: Parse CSV deliberately, inspect its contract, and preserve raw input before transformation. Then name one realistic failure mode and explain how you would prove the result using this check: For CSV properly: parse, inspect, preserve raw: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for CSV properly: parse, inspect, preserve raw with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-02.

DE05-02 - JSON and JSONL from zero

1. Why it matters

Read and write JSON and JSONL while explaining their different record shapes. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Read and write JSON and JSONL while explaining their different record shapes.

3. Prerequisite / what you already know

Prerequisite: DE05-01 mastered with saved evidence.

4. Picture it in your head

JSON is one nested document; JSONL is a stream of independent JSON records, one per line. The parser and validation strategy should match that physical shape.

5. New words before we use them

JSON can represent one nested value/document. JSONL stores one JSON value per line, useful for append/stream-style records and

- easy line-level recovery.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Convert six sample JSON records to JSONL and prove every line parses independently.

8. Expected result / what you should see

Every JSONL line parses; IDs/count reconcile; nested data meaning is not flattened accidentally.

9. Why it worked

This works because read and write JSON and JSONL while explaining their different record shapes is made explicit before the result is accepted. Your independent check is: For JSON and JSONL from zero: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling JSON and JSONL from zero complete because the command or tool finished. 2. Skipping the independent control: For JSON and JSONL from zero: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: JSON is one nested document; JSONL is a stream of independent JSON records, one per line. 3. Run the earliest independent control: For JSON and JSONL from zero: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create a new three-record nested JSON source, convert to JSONL, then reconstruct the list and compare IDs/counts.

13. Validate your result

For JSON and JSONL from zero: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

14. Teach it back in your own words

Without notes, explain JSON and JSONL from zero in your own words. Start with the purpose: Read and write JSON and JSONL while explaining their different record shapes. Then name one realistic failure mode and explain how you would prove the result using this check: For JSON and JSONL from zero: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for JSON and JSONL from zero with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-03.

DE05-03 - Paths, folders and deterministic batch files

1. Why it matters

Use pathlib and deterministic folder/file naming so batch runs are reproducible. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Use pathlib and deterministic folder/file naming so batch runs are reproducible.

3. Prerequisite / what you already know

Prerequisite: DE05-02 mastered with saved evidence.

4. Picture it in your head

A deterministic batch process should behave like a numbered filing system: the same input path/rule produces the same discovered file set and output location every run.

5. New words before we use them

Paths are part of pipeline behavior. Derive paths from project root/config and name run outputs from explicit business/run keys, not

- random Downloads locations.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Build `batch_path(run_date, source_name)` and show the same inputs produce the same path.

8. Expected result / what you should see

Same logical run keys produce same intended paths; no personal absolute path is required.

9. Why it worked

The setup action is trustworthy because the actual file/system state is checked after the operation; successful clicking or command execution alone is not treated as proof. Your independent check is: For Paths, folders and deterministic batch files: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling Paths, folders and deterministic batch files complete because the command or tool finished. 2. Skipping the independent control: For Paths, folders and deterministic batch files: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: A deterministic batch process should behave like a numbered filing system: the same input path/rule produces the same discovered file set and output location every run. 3. Run the earliest independent control: For Paths, folders and deterministic batch files: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

For a new source and run date, create the expected landing/output paths from config and prove rerunning path construction makes no duplicate/random folder.

13. Validate your result

For Paths, folders and deterministic batch files: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed.

14. Teach it back in your own words

Without notes, explain Paths, folders and deterministic batch files in your own words. Start with the purpose: Use pathlib and deterministic folder/file naming so batch runs are reproducible. Then name one realistic failure mode and explain how you would prove the result using this check: For Paths, folders and deterministic batch files: Verify the exact path, filename/extension, byte size or hash where appropriate, and confirm the original/working-copy rule was followed.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Paths, folders and deterministic batch files with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-04.

DE05-04 - pandas profiling before transformation

1. Why it matters

Profile a pandas batch before transforming it. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Profile a pandas batch before transforming it.

3. Prerequisite / what you already know

Prerequisite: DE05-03 mastered with saved evidence.

4. Picture it in your head

Profile before transformation like a doctor checking vitals before treatment: shape, types, nulls, duplicates, ranges and key uniqueness tell you what is actually wrong.

5. New words before we use them

Profiling answers what arrived: rows, columns, types, missingness, candidate-key uniqueness and simple ranges.
Transformation should

- start only after these controls are recorded.

6. Useful video / source

Source: Luke Barousse — pandas / Python mechanics refresher

Assigned scope: 4:22:56–5:08:26

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=wUSDVGivd-8)

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Create a profiling dictionary and save it as JSON evidence.

8. Expected result / what you should see

Profile is based on observed source; row/key/null/type controls are saved separately from cleaning.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting “the script ran” as correctness. Your independent check is: For pandas profiling before transformation: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling pandas profiling before transformation complete because the command or tool finished. 2. Skipping the independent control: For pandas profiling before transformation: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Profile before transformation like a doctor checking vitals before treatment: shape, types, nulls, duplicates, ranges and key uniqueness tell you what is actually wrong. 3. Run the earliest independent control: For pandas profiling before transformation: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Profile orders_profile_variant.csv and flag at least three suspicious properties without fixing them yet.

13. Validate your result

For pandas profiling before transformation: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

14. Teach it back in your own words

Without notes, explain pandas profiling before transformation in your own words. Start with the purpose: Profile a pandas batch before transforming it. Then name one realistic failure mode and explain how you would prove the result using this check: For pandas profiling before transformation: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for pandas profiling before transformation with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-05.

DE05-05 - Cleaning with pandas and rejected rows

1. Why it matters

Clean with pandas while preserving rejected rows and explicit reasons. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Clean with pandas while preserving rejected rows and explicit reasons.

3. Prerequisite / what you already know

Prerequisite: DE05-04 mastered with saved evidence.

4. Picture it in your head

Picture cleaning as two lanes: accepted rows move forward only after rules pass; rejected rows go to quarantine with a reason instead of disappearing silently.

5. New words before we use them

A production-minded cleaning step should separate accepted and rejected data. Bad rows are evidence, not trash. Attach reason codes

- rather than silently dropping.

6. Useful video / source

Source: Luke Barousse — pandas / Python mechanics refresher

Assigned scope: 4:22:56–5:08:26

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=wUSDVGivd-8)

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Process orders_bad.csv so every invalid row gets one or more reason codes.

8. Expected result / what you should see

All source rows accounted for; rejects retain source fields + reason; no bad row silently disappears.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting “the script ran” as correctness. Your independent check is: For Cleaning with pandas and rejected rows: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling Cleaning with pandas and rejected rows complete because the command or tool finished. 2. Skipping the independent control: For Cleaning with pandas and rejected rows: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture cleaning as two lanes: accepted rows move forward only after rules pass; rejected rows go to quarantine with a reason instead of disappearing silently. 3. Run the earliest independent control: For Cleaning with pandas and rejected rows: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

On a changed bad batch, produce accepted + rejected datasets and reconcile $\text{source_rows} = \text{accepted_rows} + \text{rejected_rows}$.

13. Validate your result

For Cleaning with pandas and rejected rows: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

14. Teach it back in your own words

Without notes, explain Cleaning with pandas and rejected rows in your own words. Start with the purpose: Clean with pandas while preserving rejected rows and explicit reasons. Then name one realistic failure mode and explain how you would prove the result using this check: For Cleaning with pandas and rejected rows: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Cleaning with pandas and rejected rows with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-06.

DE05-06 - pandas merge, cardinality and aggregation

1. Why it matters

Merge pandas tables with cardinality checks and aggregate without fan-out. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Merge pandas tables with cardinality checks and aggregate without fan-out.

3. Prerequisite / what you already know

Prerequisite: DE05-05 mastered with saved evidence.

4. Picture it in your head

Picture one customer card connected to many order cards. Cardinality predicts how many matches each side can produce; unexpected one-to-many links create fan-out and multiply rows.

5. New words before we use them

Before merge, state left/right grain and prove uniqueness on the intended one side. `validate=` in pandas merge can enforce expected

- cardinality.

6. Useful video / source

Source: Luke Barousse — pandas / Python mechanics refresher

Assigned scope: 4:22:56–5:08:26

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=wUSDVGivd-8)

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Run the valid merge, then deliberately duplicate a customer key and observe the validation failure.

8. Expected result / what you should see

Pre/post row counts preserved when expected; duplicate one-side key blocks the merge; unmatched count is explicit; grouped total

- reconciles.

9. Why it worked

The relationship is trustworthy because the join key and expected cardinality are stated before interpreting the output, and row/total controls can expose fan-out or dropped unmatched rows. Your independent check is: For pandas merge, cardinality and aggregation: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling pandas merge, cardinality and aggregation complete because the command or tool finished. 2. Skipping the independent control: For pandas merge, cardinality and aggregation: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture one customer card connected to many order cards. 3. Run the earliest independent control: For pandas merge, cardinality and aggregation: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Use a changed dimension with one missing and one duplicated key; diagnose both without using drop_duplicates as a blind fix.

13. Validate your result

For pandas merge, cardinality and aggregation: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation.

14. Teach it back in your own words

Without notes, explain pandas merge, cardinality and aggregation in your own words. Start with the purpose: Merge pandas tables with cardinality checks and aggregate without fan-out. Then name one realistic failure mode and explain how you would prove the result using this check: For pandas merge, cardinality and aggregation: Check pre/post row counts, key uniqueness on the expected “one” side, unmatched-key counts, and one important total before and after the relationship operation.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for pandas merge, cardinality and aggregation with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-07.

DE05-07 - Data validation: record, batch and relationship rules

1. Why it matters

Implement record, batch and relationship validation as explicit contracts. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Implement record, batch and relationship validation as explicit contracts.

3. Prerequisite / what you already know

Prerequisite: DE05-06 mastered with saved evidence.

4. Picture it in your head

Think of validation at three zoom levels: one record, the whole batch, and relationships between tables. A batch can pass row rules but still fail totals or referential integrity.

5. New words before we use them

Validation has layers: record field/type/range rules; batch row/key/freshness rules; relationship rules against reference data. Collect

- actionable failures before publish.

6. Useful video / source

Source: Corey Schafer — Complete Pydantic Data Validation Course

Assigned scope: Validation fundamentals → field/custom/nested validation; use only sections relevant to pipeline records

How to use it: Use this modern validation class for the mental model, then implement the course record/batch/relationship validation rules independently.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=M81pfi64eeM)

URL: <https://www.youtube.com/watch?v=M81pfi64eeM>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Validate good data and a packaged bad batch. Produce reason-coded error output.

8. Expected result / what you should see

Good data passes; each controlled defect is detected; validation report identifies row/key/rule; publish is blocked on critical failure.

9. Why it worked

This works because implement record, batch and relationship validation as explicit contracts is made explicit before the result is accepted. Your independent check is: For Data validation: record, batch and relationship rules: record the actual before/after state and verify the lesson requirement with an independent control. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling Data validation: record, batch and relationship rules complete because the command or tool finished. 2. Skipping the independent control: For Data validation: record, batch and relationship rules: record the actual before/after state and verify the lesson requirement with an independent control. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Think of validation at three zoom levels: one record, the whole batch, and relationships between tables. 3. Run the earliest independent control: For Data validation: record, batch and relationship rules: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Add a new rule (allowed status or max future date) and prove a violating record is rejected while valid records still pass.

13. Validate your result

For Data validation: record, batch and relationship rules: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Good data passes; each controlled defect is detected; validation report identifies row/key/rule; publish is blocked on critical failure.

14. Teach it back in your own words

Without notes, explain Data validation: record, batch and relationship rules in your own words. Start with the purpose: Implement record, batch and relationship validation as explicit contracts. Then name one realistic failure mode and explain how you would prove the result using this check: For Data validation: record, batch and relationship rules: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Data validation: record, batch and relationship rules with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-08.

DE05-08 - HTTP and REST APIs from zero

1. Why it matters

Explain HTTP/REST request-response basics using a safe local training API. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Explain HTTP/REST request-response basics using a safe local training API.

3. Prerequisite / what you already know

Prerequisite: DE05-07 mastered with saved evidence.

4. Picture it in your head

An API call is a request/response envelope: method + URL + headers + parameters go out; status + headers + body come back. Treat the response contract as data, not magic.

5. New words before we use them

HTTP exchanges a request for a response. Method, URL/path, query parameters, headers, status and body are separate parts.

- 2xx/4xx/5xx categories guide handling.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Call GET /health and GET /orders?page=1&per_page=4; explain path vs query parameter.

8. Expected result / what you should see

200 payload shape is understood; 404 is not parsed as valid orders; 500 is classified as server failure.

9. Why it worked

The ingestion is trustworthy because request boundaries, response status/schema and progress are checked explicitly; arrival of a response is not treated as proof that the data is complete or valid. Your independent check is: For HTTP and REST APIs from zero: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling HTTP and REST APIs from zero complete because the command or tool finished. 2. Skipping the independent control: For HTTP and REST APIs from zero: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: An API call is a request/response envelope: method + URL + headers + parameters go out; status + headers + body come back. 3. Run the earliest independent control: For HTTP and REST APIs from zero: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Call /missing and /orders?fail=1; classify each result and state whether pipeline should retry, stop or treat as valid.

13. Validate your result

For HTTP and REST APIs from zero: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated.

14. Teach it back in your own words

Without notes, explain HTTP and REST APIs from zero in your own words. Start with the purpose: Explain HTTP/REST request-response basics using a safe local training API. Then name one realistic failure mode and explain how you would prove the result using this check: For HTTP and REST APIs from zero: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for HTTP and REST APIs from zero with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-09.

DE05-09 - Calling APIs safely with requests

1. Why it matters

Call APIs safely with requests using timeout, status handling and response validation. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Call APIs safely with requests using timeout, status handling and response validation.

3. Prerequisite / what you already know

Prerequisite: DE05-08 mastered with saved evidence.

4. Picture it in your head

Picture an API client as a cautious courier: set timeouts, check status, parse only expected content, log the request context, and never assume a response is valid because it arrived.

5. New words before we use them

Network calls can fail slowly or return invalid payloads. A safe client uses timeouts, status handling, schema checks and bounded useful

- logging.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Implement `fetch_page()` for local API and test 200/404/500 behavior.

8. Expected result / what you should see

Timeout configured; non-2xx does not enter normal processing; payload contract failures are distinct from HTTP failures.

9. Why it worked

The ingestion is trustworthy because request boundaries, response status/schema and progress are checked explicitly; arrival of a response is not treated as proof that the data is complete or valid. Your independent check is: For Calling APIs safely with requests: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling APIs safely with requests complete because the command or tool finished. 2. Skipping the independent control: For Calling APIs safely with requests: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture an API client as a cautious courier: set timeouts, check status, parse only expected content, log the request context, and never assume a response is valid because it arrived. 3. Run the earliest independent control: For Calling APIs safely with requests: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Point `fetch_page` at a changed endpoint response missing expected items; fail clearly rather than returning empty success.

13. Validate your result

For Calling APIs safely with requests: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated.

14. Teach it back in your own words

Without notes, explain Calling APIs safely with requests in your own words. Start with the purpose: Call APIs safely with requests using timeout, status handling and response validation. Then name one realistic failure mode and explain how you would prove the result using this check: For Calling APIs safely with requests: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Calling APIs safely with requests with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-10.

DE05-10 - Pagination and authentication without leaking secrets

1. Why it matters

Handle pagination and authentication without leaking secrets. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Handle pagination and authentication without leaking secrets.

3. Prerequisite / what you already know

Prerequisite: DE05-09 mastered with saved evidence.

4. Picture it in your head

Pagination is turning through numbered pages until the server says there are no more; authentication is showing a credential at the door. The secret belongs in configuration, not source code/logs.

5. New words before we use them

Pagination is state: page/cursor advances only after a successful validated response. Authentication belongs in environment/secret

- storage, never source code or screenshots.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Fetch all local API pages with TRAINING_TOKEN supplied via environment. Save page/count evidence, not the token.

8. Expected result / what you should see

All expected IDs fetched once; token absent from source/log; auth failure stops clearly; pagination termination is deterministic.

9. Why it worked

The ingestion is trustworthy because request boundaries, response status/schema and progress are checked explicitly; arrival of a response is not treated as proof that the data is complete or valid. Your independent check is: For Pagination and authentication without leaking secrets: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling Pagination and authentication without leaking secrets complete because the command or tool finished. 2. Skipping the independent control: For Pagination and authentication without leaking secrets: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or dupl. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Pagination is turning through numbered pages until the server says there are no more; authentication is showing a credential at the door. 3. Run the earliest independent control: For Pagination and authentication without leaking secrets: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Change per_page and repeat; total IDs must remain the same. Then test missing/invalid token and prove no secret is logged.

13. Validate your result

For Pagination and authentication without leaking secrets: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated.

14. Teach it back in your own words

Without notes, explain Pagination and authentication without leaking secrets in your own words. Start with the purpose: Handle pagination and authentication without leaking secrets. Then name one realistic failure mode and explain how you would prove the result using this check: For Pagination and authentication without leaking secrets: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Pagination and authentication without leaking secrets with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-11.

DE05-11 - Retries, rate limits and partial failure

1. Why it matters

Design bounded retries for rate limits/transient failures and preserve partial-failure evidence. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Design bounded retries for rate limits/transient failures and preserve partial-failure evidence.

3. Prerequisite / what you already know

Prerequisite: DE05-10 mastered with saved evidence.

4. Picture it in your head

Retries are a controlled second attempt, not an infinite loop. Backoff spaces attempts, rate limits protect the service, and partial failures must be recorded so already-successful work is not duplicated.

5. New words before we use them

Retry only failures that may recover, bound attempts, honor Retry-After/backoff, and never retry permanent data-contract errors blindly.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Use local ?rate_limit=1 and ?fail_once=1 training cases; observe bounded recovery/failure.

8. Expected result / what you should see

Retries bounded; 429/5xx policy explicit; partial completion state distinguishable from full success; no infinite loop.

9. Why it worked

The ingestion is trustworthy because request boundaries, response status/schema and progress are checked explicitly; arrival of a response is not treated as proof that the data is complete or valid. Your independent check is: For Retries, rate limits and partial failure: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling Retries, rate limits and partial failure complete because the command or tool finished. 2. Skipping the independent control: For Retries, rate limits and partial failure: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Retries are a controlled second attempt, not an infinite loop. 3. Run the earliest independent control: For Retries, rate limits and partial failure: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Simulate a page that fails permanently after earlier pages succeeded; preserve last safe page and do not falsely mark full batch complete.

13. Validate your result

For Retries, rate limits and partial failure: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated.

14. Teach it back in your own words

Without notes, explain Retries, rate limits and partial failure in your own words. Start with the purpose: Design bounded retries for rate limits/transient failures and preserve partial-failure evidence. Then name one realistic failure mode and explain how you would prove the result using this check: For Retries, rate limits and partial failure: Save status code, response/page count, record count and final pagination/checkpoint state; retry a controlled failure and prove no page is lost or duplicated.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Retries, rate limits and partial failure with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-12.

DE05-12 - Python + PostgreSQL with SQLAlchemy

1. Why it matters

Use SQLAlchemy to connect Python to the already-installed PostgreSQL lab safely. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Use SQLAlchemy to connect Python to the already-installed PostgreSQL lab safely.

3. Prerequisite / what you already know

Prerequisite: DE05-11 mastered with saved evidence.

4. Picture it in your head

Picture SQLAlchemy/DB access as a managed bridge: configuration opens a connection, a transaction carries statements across, and explicit commit/rollback plus row controls prove what changed.

5. New words before we use them

SQLAlchemy manages engine/connection and parameterized execution; PostgreSQL was first installed/verified in M03. M05 teaches

- Python integration, not database modeling.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Load a small accepted batch into a dedicated m05 schema/table, then query row count and total; rerun with stable key/upsert or clean-reset strategy stated explicitly.

8. Expected result / what you should see

No credential in code; connection works; row/key/totals reconcile; failure rolls back; rerun policy is explicit.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting “the script ran” as correctness. Your independent check is: For Python + PostgreSQL with SQLAlchemy: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling Python + PostgreSQL with SQLAlchemy complete because the command or tool finished. 2. Skipping the independent control: For Python + PostgreSQL with SQLAlchemy: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture SQLAlchemy/DB access as a managed bridge: configuration opens a connection, a transaction carries statements across, and explicit commit/rollback plus row controls prove what changed. 3. Run the earliest independent control: For Python + PostgreSQL with SQLAlchemy: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Load a changed batch using a separate run_id. Prove a failed transaction does not leave half-published rows.

13. Validate your result

For Python + PostgreSQL with SQLAlchemy: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

14. Teach it back in your own words

Without notes, explain Python + PostgreSQL with SQLAlchemy in your own words. Start with the purpose: Use SQLAlchemy to connect Python to the already-installed PostgreSQL lab safely. Then name one realistic failure mode and explain how you would prove the result using this check: For Python + PostgreSQL with SQLAlchemy: Query the database/schema context, inspect the created object/constraint, and run one positive plus one deliberately invalid case to prove the rule is actually enforced.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Python + PostgreSQL with SQLAlchemy with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-13.

DE05-13 - Configuration, environment variables and secret hygiene

1. Why it matters

Separate configuration, environment-specific values and secrets from logic. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Separate configuration, environment-specific values and secrets from logic.

3. Prerequisite / what you already know

Prerequisite: DE05-12 mastered with saved evidence.

4. Picture it in your head

Configuration is the set of knobs that may change by environment; secrets are locked keys. Keep both outside hard-coded business logic so the same code can move safely.

5. New words before we use them

Config is data controlling behavior. Non-secret defaults/examples can live in repo; secrets come from environment/secret manager and

- must never be committed.

6. Useful video / source

Source: Luke Barousse — environments / project setup

Assigned scope: 5:50:56–6:27:15

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=wUSDVGivd-8)

URL: <https://www.youtube.com/watch?v=wUSDVGivd-8>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Override output directory and DATABASE_URL via environment without editing Python.

8. Expected result / what you should see

Same code runs with two configs; missing required setting fails clearly; secrets are excluded from Git/log evidence.

9. Why it worked

This works because separate configuration, environment-specific values and secrets from logic is made explicit before the result is accepted. Your independent check is: For Configuration, environment variables and secret hygiene: record the actual before/after state and verify the lesson requirement with an independent control. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling Configuration, environment variables and secret hygiene complete because the command or tool finished. 2. Skipping the independent control: For Configuration, environment variables and secret hygiene: record the actual before/after state and verify the lesson requirement with an independent control. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Configuration is the set of knobs that may change by environment; secrets are locked keys. 3. Run the earliest independent control: For Configuration, environment variables and secret hygiene: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Create a second config profile with different batch size/output path; prove code unchanged and secret value never printed.

13. Validate your result

For Configuration, environment variables and secret hygiene: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Same code runs with two configs; missing required setting fails clearly; secrets are excluded from Git/log evidence.

14. Teach it back in your own words

Without notes, explain Configuration, environment variables and secret hygiene in your own words. Start with the purpose: Separate configuration, environment-specific values and secrets from logic. Then name one realistic failure mode and explain how you would prove the result using this check: For Configuration, environment variables and secret hygiene: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Configuration, environment variables and secret hygiene with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-14.

DE05-14 - Logging, testing and reusable pipeline code

1. Why it matters

Combine structured logging, tests and reusable pipeline code around clear boundaries. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Combine structured logging, tests and reusable pipeline code around clear boundaries.

3. Prerequisite / what you already know

Prerequisite: DE05-13 mastered with saved evidence.

4. Picture it in your head

Logs are the flight recorder, tests are pre-flight checks, and reusable functions are replaceable components. Together they make failures diagnosable instead of mysterious.

5. New words before we use them

Reliable code separates pure transformations from I/O, logs run context/counts, and tests both small functions and integrated behavior.

6. Useful video / source

Source: PyCon US 2025 — Building Resilient Data Pipelines: The Power of Idempotency

Assigned scope: Full ~25-minute talk; focus on resilient pipeline design + pytest strategy

How to use it: Use this as the testing/reliability visual layer. The Teaching Book still teaches logging and reusable code structure. Close the video before the independent pipeline task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=kT8YWX4NE2o)

URL: <https://www.youtube.com/watch?v=kT8YWX4NE2o>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Run tests, then inject one bad record and confirm the expected test/failure evidence.

8. Expected result / what you should see

Tests fail before repair and pass after; logs identify run and counts; functions have clear responsibilities; failures propagate.

9. Why it worked

This works because combine structured logging, tests and reusable pipeline code around clear boundaries is made explicit before the result is accepted. Your independent check is: For Logging, testing and reusable pipeline code: record the actual before/after state and verify the lesson requirement with an independent control. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling Logging, testing and reusable pipeline code complete because the command or tool finished. 2. Skipping the independent control: For Logging, testing and reusable pipeline code: record the actual before/after state and verify the lesson requirement with an independent control. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Logs are the flight recorder, tests are pre-flight checks, and reusable functions are replaceable components. 3. Run the earliest independent control: For Logging, testing and reusable pipeline code: record the actual before/after state and verify the lesson requirement with an independent control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Add one regression test for a bug you deliberately create and repair. Explain what the test prevents.

13. Validate your result

For Logging, testing and reusable pipeline code: record the actual before/after state and verify the lesson requirement with an independent control. Challenge this expected outcome directly: Tests fail before repair and pass after; logs identify run and counts; functions have clear responsibilities; failures propagate.

14. Teach it back in your own words

Without notes, explain Logging, testing and reusable pipeline code in your own words. Start with the purpose: Combine structured logging, tests and reusable pipeline code around clear boundaries. Then name one realistic failure mode and explain how you would prove the result using this check: For Logging, testing and reusable pipeline code: record the actual before/after state and verify the lesson requirement with an independent control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Logging, testing and reusable pipeline code with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-15.

DE05-15 - Integrated Python data-engineering pipeline

1. Why it matters

Build and defend an integrated Python ingestion pipeline with safe reruns and PostgreSQL/output evidence. This matters because data-engineering Python must preserve source evidence and make bad inputs visible instead of silently rewriting or dropping them.

2. What you are learning

Build and defend an integrated Python ingestion pipeline with safe reruns and PostgreSQL/output evidence.

3. Prerequisite / what you already know

Prerequisite: DE05-14 mastered with saved evidence.

4. Picture it in your head

Picture the whole pipeline as receipt -> validate -> transform -> load -> reconcile -> log. Each arrow has an evidence checkpoint so reruns and failures are explainable.

5. New words before we use them

The final module proof combines preserve raw -> profile -> validate/quarantine -> transform -> API/file ingestion -> safe state -> optional

- PostgreSQL load -> logs/tests -> reconciliation.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[OPEN VIDEO / SOURCE - CLICK HERE \(youtube.com\)](https://www.youtube.com/watch?v=IP8xXebHmuE)

URL: <https://www.youtube.com/watch?v=IP8xXebHmuE>

Do this after watching

Close the external source. Return to the bundled lesson and perform the exact Follow with me once steps from your own keyboard. Save evidence from your course data/tool, not a screenshot of the video.

7. Follow with me once

Build one file-source run and one API-source run. Save source receipt, accepted/rejected counts, output/database controls, log and rerun result.

8. Expected result / what you should see

PASS only when all integrated pipeline contracts are evidenced: raw/source receipt preserved; accepted + rejected reconcile to input; no

- silent bad-data publish; rerun creates no duplicate publish; transient failure follows bounded recovery; permanent/partial failure is not
- labeled full success; no secret appears in code/log/evidence; output or PostgreSQL state reconciles; tests/logs support the run; and
- another learner could reproduce the run from README/config. Any unresolved critical contract => TARGETED REPAIR.

9. Why it worked

The program is trustworthy because input/state changes are explicit and the output is checked against a known condition rather than accepting “the script ran” as correctness. Your independent check is: For Integrated Python data-engineering pipeline: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. The proof is source identity plus input/output controls and rerun behavior, not a green script run.

10. Common beginner mistakes

1. Calling Integrated Python data-engineering pipeline complete because the command or tool finished. 2. Skipping the independent control: For Integrated Python data-engineering pipeline: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 3. Changing several things after a failure instead of isolating the first broken step. 4. Overwriting the failed attempt instead of saving it as repair evidence.

11. If you get stuck - recovery ladder

1. Return to the lesson prerequisite/starting state; do not stack fixes on an unknown state. 2. Rebuild the mental picture: Picture the whole pipeline as receipt -> validate -> transform -> load -> reconcile -> log. 3. Run the earliest independent control: For Integrated Python data-engineering pipeline: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control. 4. Repair only the first failed step, rerun that same control, and save the failed attempt before opening protected Review.

12. Now you do a small version alone

Run a fresh changed batch with one duplicate, one invalid row and one transient API failure. Recover without corrupting previously published data.

13. Validate your result

For Integrated Python data-engineering pipeline: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

14. Teach it back in your own words

Without notes, explain Integrated Python data-engineering pipeline in your own words. Start with the purpose: Build and defend an integrated Python ingestion pipeline with safe reruns and PostgreSQL/output evidence. Then name one realistic failure mode and explain how you would prove the result using this check: For Integrated Python data-engineering pipeline: Run the independent input from a clean state, record the actual output/count/schema/hash or exception, and compare it with an independently stated expected control.

15. Protected review + changed fresh retry

Save the genuine independent attempt first. If it is weak, open only the matching protected Review, identify the exact gap, record the repair, then close Review. Do a changed retry for Integrated Python data-engineering pipeline with a different input/identifier/condition and validate it independently. Work completed while Review is open is repair evidence, not mastery evidence.

16. Evidence / mastery + exact next route

Mastery evidence: Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: module practical / Gate route after the mastery checklist.